

Manuale d'uso — correlazioni dinamiche trimero anello

versione a stato esatto (`prepare_state`)

Note di lavoro — Tesi triennale, Samuele Schiavi
Relatore: Prof. Alessandro Chiesa

Scopo

Questo manuale spiega come eseguire la simulazione delle correlazioni dinamiche $C_{ij}^{\alpha\beta}(t)$ per il trimero ad anello nella sua versione *originale*: lo stato fondamentale ψ_0 è preparato sul registro con le ampiezze esatte (`QuantumCircuit.prepare_state`), non con un ansatz VQE. È la versione da usare quando interessa isolare l'errore di Trotter e quello statistico (shot), senza mescolarli con un residuo di fedeltà dell'ansatz variazionale. Per la versione con il circuito VQE reale (W-2q.6), vedi il manuale gemello `manuale_uso_correlazioni_trimero_anello_vqe.tex`.

Prerequisiti

File necessari nella stessa cartella (tutti nella fase *correlazioni dinamiche*, trimero anello):

- `trimer_ring_exact.py` — Hamiltoniana `trimer_hamiltonian_dm`.
- `trotter_trimero_anello.py` — evoluzione $U(t) = e^{-iHt}$ via Suzuki–Trotter.
- `circuito_correlazioni_trimero_anello.py` — il circuito Hadamard test a 4 qubit.
- `validate_circuito_correlazioni.py` — riferimento classico esatto (`scipy.linalg.expm`).
- `validate_shot_noise_trimero_anello.py` — validazione a shot finiti.
- `scan81_trimero_anello.py` — scan completo delle 81 combinazioni.
- `generate_figure_shotnoise.py`, `generate_figure_scan81.py` — figure.

Librerie Python: `qiskit` (2.x), `qiskit-aer`, `numpy`, `scipy`, `matplotlib` (solo per le figure).
Verificare con:

```
python -c "import qiskit; print(qiskit.__version__)"
```

Punto di lavoro di riferimento (usato ovunque in questa fase, salvo diversa indicazione):
 $J = 1$, $J' = 0.4$, $b = b_c = 2.4$, $D = 0.15$ (DM, Opzione B), $t = 1.3$, $N = 200$ passi di Trotter.

Passo 1 — un correlatore singolo, da riga di comando Python

Per calcolare un correlatore specifico, ad esempio $C_{11}^{yy}(t=1.3)$:

```
from circuito_correlazioni_trimero_anello import ground_state, correlator_from_circuit
```

```
J, Jp, b, D = 1.0, 0.4, 2.4, 0.15  
psi0, E = ground_state(J, Jp, b, D)
```

```
C = correlator_from_circuit(1, 'y', 1, 'y', t=1.3, N=200, J=J, Jp=Jp, b=b, D=D, psi0=psi0)  
print(C)    # atteso: un numero complesso, |C| dell'ordine di alcuni decimi
```

Non passare `ansatz_params`: di default resta `None` e la preparazione avviene con `prepare_state(psi0)`, ampiezze esatte.

Passo 2 — validazione contro il riferimento classico esatto

```
python validate_circuito_correlazioni.py
```

Stampa un confronto, per sei correlatori rappresentativi e tre valori di t , tra il valore classico esatto (`scipy.linalg.expm`, nessun errore di Trotter) e quello dal circuito ($N = 200$): il residuo atteso è dell'ordine di 10^{-5} – 10^{-6} (errore di Trotter puro). In coda, uno studio di convergenza in N su $C_{11}^{yy}(t=1.3)$: l'errore deve dimezzarsi raddoppiando N (Trotter al primo ordine, $O(1/N)$ sull'ampiezza).

Passo 3 — validazione a shot finiti

```
python validate_shot_noise_trimer_o_anello.py
```

Separa i due contributi di errore: quello algoritmico (Trotter, statevector vs esatto classico) e quello statistico (shot vs statevector, 8192 shots). Al punto di lavoro standard l'errore di Trotter domina nettamente su quello statistico (rapporto $\sim 16\times$, vedi `circuito_correlazioni_trimer_o_anello_spiegato.tex`) – regime opposto a quello del dimero.

Passo 4 — scan completo delle 81 combinazioni

```
python scan81_trimer_o_anello.py
```

Richiede qualche minuto (81 combinazioni $\times 2$ parti Re/Im $\times$ (statevector + 8192 shots), con cache di trascompilazione). Stampa un riepilogo (zeri strutturali confermati, errori medi/massimi, correlatori più "ricchi") e salva due file nella cartella corrente:

- `scan81_results.npz` — matrici 9×9 (una riga/colonna per ogni coppia sito-componente) di riferimento classico, statevector e shot, più le statistiche di errore. Si carica con `numpy.load(...)`.
- `scan81_results.json` — stessi dati, riga per riga, in formato leggibile/ispezionabile a mano.

Attenzione: lo script salva con percorso *relativo* — va eseguito da dentro la cartella dove si vogliono i risultati (non da un'altra posizione), altrimenti Python solleva `FileNotFoundError`.

Passo 5 — figure

```
python generate_figure_shotnoise.py
python generate_figure_scan81.py
```

Il secondo richiede che `scan81_results.npz` sia già presente (Passo 4 completato). Producono i PNG usati nei documenti di teoria/spiegazione.

Problemi comuni

- `ModuleNotFoundError` su uno dei moduli locali — verificare di eseguire dalla cartella che contiene tutti i file elencati in "Prerequisiti", non da una sottocartella o da altrove.
- `FileNotFoundError` su `scan81_results.npz` in uno script successivo — va prima generato eseguendo `scan81_trimero_anello.py` (Passo 4); non è incluso di default perché il calcolo richiede qualche minuto.
- Risultati leggermente diversi da quelli riportati nei documenti di teoria — controllare punto di lavoro (J, J', b, D) e t, N : sono hard-coded in testa a ogni script, non parametri da riga di comando.

Riferimenti

Teoria e derivazioni: `simmetrie_correlatori_trimero_anello.tex`, `circuito_correlazioni_trimero_anello_spiegato.tex`. Catalogo completo della fase: `trimero_anello_correlazioni.tex`. Cronologia delle decisioni: `log_decisioni.md`.